

NBKR INSTITUTE OF SCIENCE AND TECHNOLOGY

USER LOGIN AND PROFILE SYSTEM

Course: Data structures

Department: Computer science

Section: E

Year: I B.tech

Semester: II

Date: April 30,2025

Submitted to: Ashok selva Kumar Sir

SUBMITTED BY:

V.Sireesha(24KB1A05MC)

U.Mounika(24KB1A05LZ)

V.Sonali(24KB1A05MN)

1. Acknowledgement:

I would like to express my sincere gratitude for the support and guidance provided during the development of this project. The user login and profile system in C was created as a simple demonstration of file handling, user authentication, and structured programming in the C language.

This project helped enhance my understanding of key programming concepts such as:

- Structs for storing user data
- File operations (reading, writing, and appending)
- String handling and validation
- Loop and control structures for menu-based programs

2. Abstract:

This project presents a simple User Login and Profile Management System developed in the C programming language. The system allows users to register with their name, email, and password, which are then stored in a text file for persistence. Registered users can log in using their credentials, and the system validates them by reading from the file. Additionally, the system provides an option to display all registered users, showcasing basic user profile handling. This project demonstrates key programming concepts in C such as file handling, structures, string manipulation, and menu-driven interfaces, making it suitable for educational purposes and beginner-level projects. Here's a shorter version of the introduction (5–6 lines):

3. Introduction:

This project is a simple **User Login and Profile System** developed in C. It allows users to register with their name, email, and password, and stores the data in a text file. Users can log in using their credentials, and the system verifies them by reading from the file. It also includes a feature to display all registered users. The project demonstrates basic C programming concepts such as **structures**, **file handling**, and **menu-driven interfaces**. It is ideal for beginners learning how to manage user data and implement authentication.

4.Objective:

- To develop a basic user login and registration system using the C programming language.
- To implement user data storage and retrieval using file handling techniques.
- To practice the use of structures for organizing user information.
- To create a menu-driven interface for easy user interaction.
- To validate user credentials securely during the login process.
- To provide a simple method for viewing all registered users.

5.Software Requirements:

Hardware Requirements:

Processor: Intel Pentium or higher

RAM: Minimum 512 MB

Hard Disk: Minimum 10 MB of free space

Monitor, Keyboard, and Mouse for interaction

Software Requirements:

Operating System: Windows, Linux, or macOS

Compiler: GCC (MinGW for Windows or GCC on Linux/macOS)

Text Editor: Code::Blocks, Dev-C++, VS Code, or any C-compatible IDE

Terminal or Command Prompt for compiling and executing the program

6.Methodology:

The development of the User Login and Profile System in C followed a structured, step-by-step approach:

1. **Requirement Analysis:** Identified the need for basic user registration, login, and data storage.
2. **Design Phase:** Defined the structure for user data using a `struct`, and designed a menu-driven interface for user interaction.
3. **Implementation:**
 - Used `scanf()` and `fprintf()` to take input and save user data to a text file.
 - Used `fscanf()` to read and verify user login credentials.
 - Applied string comparison functions to validate input.
4. **Testing:** Verified all functionalities—registration, login, and user display—using various test cases.
5. **Finalization:** Ensured the program handles invalid inputs gracefully and provides clear feedback to the user.

7.Project Description:

This project is a basic User Login and Profile System developed in the C programming language. It allows users to register with their name, email, and password, which are stored in a text file. Users can log in using their email and password, and the system verifies their credentials by reading from the file. The program also provides a feature to display all registered users. It demonstrates key concepts such as file handling, structures, and menu-driven programming in C. The system is suitable for learning how to manage simple us

8.Algorithm:

1. Start
2. Display the main menu with options:

Register

Login

Show All Users

Exit

3. If user selects Register:
 - a. Prompt for name, email, and password
 - b. Open the file in append mode
 - c. Write the user data to the file
 - d. Close the file and display success message
4. If user selects Login:
 - a. Prompt for email and password
 - b. Open the file in read mode
 - c. Read each line and compare email and password
 - d. If match found, display welcome message
 - e. If no match, display error message
 - f. Close the file.
5. If user selects Show All Users:

- a. Open the file in read mode
 - b. Read and display all user names and emails
 - c. Close the file
6. If user selects Exit:
- a. Terminate the program
- 7.End.

9. Source code :

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>


#define MAX_USERS 100

#define MAX_NAME 50

#define MAX_EMAIL 50

#define MAX_PASS 20

#define FILENAME "users.txt"


Typedef struct {

    Char name[MAX_NAME];

    Char email[MAX_EMAIL];
```

```

    Char password[MAX_PASS];

} User;

Void registerUser() {

    User user;

    FILE *fp = fopen(FILENAME, "a");

    If (fp == NULL) {

        Printf("Error opening file.\n");

        Return;

    }

    Printf("Enter name: ");

    Scanf("%[^\\n]", user.name);

    Printf("Enter email: ");

    Scanf("%s", user.email);

    Printf("Enter password: ");

    Scanf("%s", user.password);

    Fprintf(fp, "%s| %s| %s\\n", user.name, user.email, user.password);

    Fclose(fp);

    Printf("User registered successfully.\n");

}

```

```

Int loginUser() {

    Char email[MAX_EMAIL], password[MAX_PASS];

    User user;

    FILE *fp = fopen(FILENAME, "r");


    If (fp == NULL) {

        Printf("Error opening file.\n");

        Return 0;

    }

    Printf("Enter email: ");

    Scanf(" %s", email);

    Printf("Enter password: ");

    Scanf(" %s", password);

    While (fscanf(fp, "[%^|]|%[^|]|%[^\\n]\\n", user.name, user.email, user.password) !=
EOF) {

        If (strcmp(email, user.email) == 0 && strcmp(password, user.password) == 0) {

            Printf("Login successful! Welcome, %s.\n", user.name);

            Fclose(fp);

            Return 1;

        }
    }
}

```



```
}
```

```
Fclose(fp);
```

```
Printf("Login failed. Invalid email or password.\n");
```

```
Return 0;
```

```
}
```

```
Void showAllUsers() {
```

```
    User user;
```

```
    FILE *fp = fopen(FILENAME, "r");
```

```
    If (fp == NULL) {
```

```
        Printf("No users registered yet.\n");
```

```
        Return;
```

```
    }
```

```
    Printf("\n--- Registered Users ---\n");
```

```
    While (fscanf(fp, "[%^]|[%^]|[%^\n]\n", user.name, user.email, user.password) !=  
    EOF) {
```

```
        Printf("Name: %s, Email: %s\n", user.name, user.email);
```

```
    }
```

```
Fclose(fp);
```

```
}
```

```

Int main() {

    Int choice;

    While (1) {

        Printf("\n1. Register\n2. Login\n3. Show All Users\n4. Exit\nChoose an option: ");

        Scanf("%d", &choice);

        Switch (choice) {

            Case 1: registerUser(); break;

            Case 2: loginUser(); break;

            Case 3: showAllUsers(); break;

            Case 4: exit(0);

            Default: printf("Invalid choice.\n");

        }

    }

    Return 0;

}

```

10. Output screenshots:

```
1. Register
2. Login
3. Show All Users
4. Exit
Choose an option: 1
Enter name: mounika
Enter email: mouni@gmail.com
Enter password: mouni123
User registered successfully.

1. Register
2. Login
3. Show All Users
4. Exit
Choose an option: 2
Enter email: mouni@gmail.com
Enter password: mouni123
Login successful! Welcome, mounika .

1. Register
2. Login
3. Show All Users
4. Exit
Choose an option: 3

--- Registered Users ---
Name: mounika , Email: mouni@gmail.com

1. Register
2. Login
3. Show All Users
4. Exit
Choose an option: 4

...Program finished with exit code 0
Press ENTER to exit console.
```

11. Testing and validation:

The system was tested using various test cases to ensure all functionalities work as expected:

1. Registration Testing

- Entered valid name, email, and password → User data saved successfully in the file.
- Entered special characters or spaces → Accepted based on input handling logic.

2. Login Testing

- Correct email and password → Successfully logged in.
- Incorrect email or password → Displayed appropriate error message.
- Empty or malformed input → Rejected or prompted again.

3. User Display Testing

- Registered multiple users and verified they appear correctly using the "Show All Users" option.

4. File Handling Validation

- Checked that the file opens correctly in read/write/append modes.
- Validated that data is not overwritten during new registrations.

All core features passed basic functional testing, and the system handled invalid inputs gracefully in most cases.

12. Limitations:

- **No password encryption:** Passwords are stored in plain text, which is not secure.
- **No duplicate email check:** The system does not prevent users from registering with the same email multiple times.
- **Basic error handling:** Limited input validation and error messages.
- **Single-user access:** The system is not designed for concurrent multi-user environments.
- **Fixed storage:** User data is stored in a simple text file without database integration.
- **No profile editing or deletion:** Users cannot update or remove their profiles once registered.

13. Future enhancement:

- **Password Encryption:** Implement encryption techniques like hashing (e.g., SHA-256) to securely store passwords.